MyCodeSchool

Code review — erika-recapitulare-vectori

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 3 — 2026-10-20 — sha f17df6a

Revizuit: `f17df6a` „Refacere tema” peste runda 2 (`bbd61db`). Fiecare constatare de mai jos a fost **reprodusă prin rulare** (un harness separat, cu un prototip pus înaintea include-urilor; codul tău nu a fost atins).

Poarta de compilare: ❌ nu trece.

```
./interclasare.h:99:5: error: use of undeclared identifier 'interclasare'
```

În Visual Studio: `C3861 'interclasare': identifier not found` . Eroarea e B1 și oprește tot proiectul, deci nici `sol39` (activ în `main`) nu rulează. Restul constatărilor le-am verificat doar în harness, nu în proiectul tău.

Ce merge (prin rulare): 7.3 pe fișă (`4 9`), 6.3 pe fișă (`2`), 5.5 (treimi cu capetele corecte). Din runda 2 s-au închis 2.4 (`>` vs `>=`), 10.5, 8.5, 14.5 și `sol12` — vezi dedesubt. Noi în runda asta: 5.3, 6.3, 7.3; B3 și B6 au apărut abia acum.

🔴 Critice

B1 — Nu compilează: `sol15` cheamă `interclasare` înainte să fie declarată

`interclasare.h:99`

CUM E ACUM: `g++ -std=c++17 -fsyntax-only *.cpp` dă `interclasare.h:99:5: error: use of undeclared identifier 'interclasare'` (inclus din `Source.cpp:9`). Separat, `recapitulare-vectori.cpp:3` dă `fatal error: 'recapitulare-vectori.h' file not found` , dar fișierul nu e în `.vcxproj` , deci în Visual Studio nu contează (vezi M8).

DE CE: Compilatorul C++ citește fișierul o singură dată, de sus în jos. Când ajunge la linia 99, știe doar de `valComune`, `sol39` și `reuniune`; `interclasare` apare abia la :108, deci la :99 numele nu există încă. Un header inclus nu schimbă nimic: `#include` lipește textul pe loc, în aceeași ordine. A doua problemă e ascunsă în spatele erorii: `sol15` e testul pentru 7.4, deci trebuia să cheme `reuniune`. Chiar dacă muți funcția mai sus și compilează, testul de la 7.4 ar verifica funcția de la 7.5.

FIX: în `sol15`: `reuniune(a, da, b, db, c, dc);` în loc de `interclasare(...)`. Regula generală: o funcție se scrie deasupra locului unde e folosită.

+ același bug la: `interclasare.h:99` este singurul loc (definiția e la :108).

B2 — 7.3: `if (c[dc] = a[i])` e atribuire; la o valoare comună 0 programul se blochează

`interclasare.h:18`

CUM E ACUM: Exemplul din fișă (`a: 1 4 7 9`, `b: 2 4 9`) dă `4 9`, corect. Dar `a: 0 1`, `b: 0 2` -> programul nu se mai oprește (oprit forțat după 3 s, exit 142). Compilatorul te-a avertizat: `interclasare.h:18: using the result of an assignment as a condition`.

DE CE: Un singur `=` nu compară, ci copiază `a[i]` în `c[dc]`, iar valoarea lui devine condiția din `if`. Orice număr diferit de 0 înseamnă `true`, de aceea pe exemplul din fișă merge din întâmplare. Când valoarea comună e 0, condiția devine `false` și `i++, j++, dc++` nu se mai execută. `i` și `j` rămân pe loc, bucla `while` vede aceleași `a[i] == b[j]` la nesfârșit. Pe ramura asta nici nu trebuie un `if`: ai ajuns acolo tocmai pentru că `a[i] == b[j]`.

FIX: pe ramura cu valori egale: `c[dc] = a[i]; dc++; i++; j++;` fără `if`.

+ același bug la: nicio altă atribuire într-o condiție (`if/while/for/&&/||`) în folder. Verificate și excluse: `frecventa-cifre.h:7, 46, 47, 50, 52` (atribuiri în corpul `if/while`, nu în condiție) și `algoritmi.h:190-210` (`c[dc] = a[i];` ca instrucțiune, corect).

B3 — 7.5: `interclasare` încă pierde dubluri (garda moștenită din `reuniune`, în cozi)

`interclasare.h:137`

CUM E ACUM: `a: 1`, `b: 2 2` -> `1 2` (corect `1 2 2`). Pe cazul din runda 2, `a: 1 3`, `b: 1 2`, `c: 3` -> `1 1 2 3` (corect `1 1 2 3 3`). Pe testul tău, `sol16`: 14 numere intră, ies 13.

DE CE: Ai separat corect 7.4 în `reuniune` (asta s-a închis), dar `interclasare` e o copie a lui `reuniune` în care ai schimbat doar bucla principală (`<=`). În cele două bucle de la coadă (:137, :147) a rămas `if (dc == 0 || c[dc-1] != ...)`, adică „nu pune dacă e egal cu ultimul". Bucla principală păstrează dublurile, cozile le aruncă, iar rezultatul depinde de locul în care cade dublura. Tot din

copiere vine ramura `else` de la :125-129, la care nu se mai ajunge: dacă `a[i] <= b[j]` e fals, atunci `b[j] <= a[i]` e sigur adevărat.

FIX: interclasarea pune tot: cozile devin `while (i < da) c[dc++] = a[i++];` și la fel pentru `b`. Ramura `else` moartă se șterge. 7.5 devine o funcție `interclasare3(a,da,b,db,d,dd,c,dc)` care cheamă `interclasare` de două ori.

+ același bug la: `interclasare.h:147` (garda `c[dc-1] != b[j]` din coada lui `b`; se scoate împreună cu :137) · `interclasare.h:122-129` (ramura moartă, dovada copierii) · `reuniune` (:73, :83) are garda doar în cozi, nu și în bucla principală — vezi B4. `algoritmi.h:180` (referința) nu are garda, deci nu e afectat.

B4 — 7.4: `reuniune` elimină dublurile tot doar la coadă (B3 din runda 2, neatins)

`interclasare.h:51`

CUM E ACUM: `a: 2 2 2`, `b: 3` -> `2 2 2 3` (corect `2 3`). Exemplul din fișă dă `1 2 4 7 9`, corect, pentru că n-are dubluri în interiorul aceluiași vector.

DE CE: În bucla principală, `c[dc++] = a[i++]` (:51) și `c[dc++] = b[j++]` (:55) copiază fără să verifice ultimul element pus. Garda `c[dc-1] != ...` există doar în cozi (:73, :83). Când `a[i] < b[j]`, un 2 urmat de alt 2 trece de două ori. Cazul „toate egale" din chenarul fișei e exact testul care prinde asta. Aici e invers față de B3: `reuniune` are nevoie de gardă peste tot, `interclasare` nicăieri. Ai pus-o în locurile greșite la amândouă.

FIX: aceeași gardă `if (dc == 0 || c[dc-1] != x)` înainte de fiecare `c[dc++] = ...` din `reuniune` (:51, :55, :58), nu doar în cozi.

+ același bug la: `interclasare.h:55` (`b: 2 2` cu `a: 1` dă `1 2 2`) · `interclasare.h:58` (ramura egalității: `a: 1 1`, `b: 1 1` dă `1 1`) · `interclasare.h:18` (`valComune` nu verifică `c[dc-1]`: `a: 1 1 1`, `b: 1 1 1` produce dubluri în intersecție — familie asemănătoare, regulă diferită).

B5 — 1.4: întoarce PRIMUL element mai mic decât maximul, nu cel mai mare dintre ele

`minime-maxime.h:148`

CUM E ACUM: `7 2 9 4 9` -> `2` (fișa: 7) · `9 2 7` -> `2` (corect 7) · `3 3 8 1` -> `3` (corect, din noroc). Funcția e rescrisă față de runda 2, dar tot greșită, acum din alt motiv.

DE CE: Pasul cu `maxVector` e bun. Problema e `ok == -1` din condiție: după ce `ok` primește prima valoare `< max`, condiția nu mai e adevărată niciodată, deci bucla se oprește practic la primul candidat. „Al doilea cel mai mare" e un maxim, doar că se ia dintre valorile `< max`. De aceea trebuie

comparat cu `ok` (`v[i] > ok`), nu oprit la primul găsit. La `3 3 8 1` primul candidat, 3, era chiar răspunsul, așa că testul a trecut. Plus: bucla pornește de la `i = 1` și sare peste `v[0]` .

FIX: `for (int i = 0; i < dim; i++) if (v[i] < max && v[i] > ok) ok = v[i];`

+ același bug la: `minime-maxime.h:146` — sentinela `ok = -1` se confundă cu un element valid negativ (și, dacă toate elementele sunt egale cu maximul, întoarce tot -1, ambiguu). Altă apariție a regulii nu există; `minVector` , `maxVector` , `pozMin` , `pozMax` sunt corecte.

B6 — 5.3: cele două jumătăți se suprapun, iar a doua citește `v[dim]` , în afara vectorului

`sortare-interval.h:28`

CUM E ACUM: Fișa: `9 7 5 3 1 2` -> `3 5 7 9 2 1` (corect `5 7 9 3 2 1`). `-1 -2 -3 -4` -> `-3 -2 0 -1` : apare un 0 care nu era în vector.

DE CE: `ordonareCrescator` și `ordonareDescrescator` folosesc `j <= stop` , deci `stop` e inclus. `sortareJumatati` le dă `(0, mij)` și `(mij, dim)` . Prima sortează 0..3, adică 4 elemente în loc de 3, iar `v[mij]` intră în ambele jumătăți. A doua sortează `mij..dim` , iar `v[dim]` e deja în afara datelor. În `sol37` vectorul e `int v[100]` , deci acolo e un 0 și ajunge în rezultat. Cu un vector declarat exact pe dimensiune, citirea iese din memoria lui. La 5.5 ai scris corect `(0, p-1), (p, 2p-1)` : acolo ai gândit capetele incluse.

FIX: `ordonareCrescator(v, 0, mij - 1); ordonareDescrescator(v, mij, dim - 1);`

+ același bug la: `sortare-interval.h:29` (`ordonareDescrescator(v, mij, dim)`) · `parcurgerea-vectoriilor.h:456` (`sol13` : `for (i = 1; i <= dim; i++)` citește `v[dim]` ; trebuie `i < dim`) · `parcurgerea-vectoriilor.h:588-589` (`sol17` : `v[i+1]` ajunge la `v[2*m]` , în afara vectorului) · `munte-vale.h:67-68` (`sortareMunte` : intervalele inclusive `(0,varf)` și `(varf,d-1)` se suprapun pe `varf`) · `sortare-interval.h:41-58` (`sortareInterval` ignoră `dim` ; un `finish >= dim-1` ar citi `v[finish+1]` în afara vectorului; apelul din `sol12` cu `1,4` e în limite).

B7 — 2.1 și 2.2: căutare binară pe vector neordonat (B5 din runda 2, neatins)

`cautare-binara.h:42`

CUM E ACUM: 2.1, exemplul din fișă: `30 12 45 7 19` , caut `19` -> `-1` (fișa: `true`). 2.2 cu `v: 19 5 14 10` , `w: 10 7 19` -> `0` (corect 2). Același vector neordonat (56, 49, 57) în testele `sol1` , `sol6` , `sol7` .

DE CE: Căutarea binară aruncă la fiecare pas o jumătate din vector, pe baza promisiunii că vectorul e ordonat. La `30 12 45 7 19` , mijlocul e `45 > 19` , deci aruncă dreapta, adică exact locul unde e 19.

Fără sortare promisiunea e falsă și algoritmul greșește fără niciun mesaj. La 2.4 ai făcut deja corect: `ceaMaiMicaValMaiMareCaX` cheamă întâi `sortare(v, dim)`. 2.1 și 2.2 au nevoie de același prim pas.

FIX: 2.1: `bool apare(v, d, x) { sortare(v, d); return cautareBinara(v, d, x) != -1; }`. La 2.2: `sortare(v, d1)` înainte de buclă. În teste folosești vectori ordonați.

+ același bug la: `cautare-binara.h:154-156` (`sol7`: `{2,23,34,56,49,57,67}` neordonat dat lui `cateInInterval(v,7,23,49)` -> `2`, corect 3) · `cautare-binara.h:40-42` (`sol1`: același vector; căutarea lui 49 întoarce -1; 23 iese corect din întâmplare) · `cautare-binara.h:119-152` (`primaPozitieMaiMareEgal`, `ultimaPozitieMaiMicaEgal`, `cateInInterval` presupun sortat și sunt corecte doar dacă apelantul sortează) · `sol6` e ok doar pentru că `ceaMaiMicaValMaiMareCaX` sortează intern (cu efect secundar asupra vectorului apelantului).

B8 — 8.4: media se calculează în `double`, dar se întoarce ca `int` (B7 din runda 2, neînchis)

`stergere.h:5`

CUM E ACUM: `1 2` -> `1 2` (corect `2`). Exemplul din fișă, `4 8 1 7` -> `8 7`, trece, pentru că media 5 e întreagă.

DE CE: Ai mutat calculul în `double` (:6, :12), dar funcția e declarată `int mediAritmetica`, iar la :17 rezultatul intră tot în `int media`. La `return`, 1.5 se trunchiază la 1, deci se pierde tocmai partea cu virgulă pe care ai calculat-o. `1 < 1` e fals și 1 nu se șterge. Tipul unei valori se păstrează doar dacă e la fel pe tot drumul: variabilă, `return`, variabila care primește rezultatul. Ajunge o singură verigă `int` ca să taie zecimalele.

FIX: `double mediAritmetica(...)` și `double media = mediAritmetica(v, dim);` la :17.

+ același bug la: `stergere.h:5` (tipul de întoarcere `int`, cauza de bază) · `:13` (`return media;`, conversie double->int) · `:17` (`int media = ...`) · `:20` (`v[i] < media` compară cu valoarea trunchiată). Nicio altă medie în repo; `frecventa-valori.h:31` (`d / 2`) e corectă.

B9 — 6.4 și căutarea binară sunt tot copiate din `algoritmi.h` (B1 din runda 2, neatins)

`munte-vale.h:29`

CUM E ACUM: `diff` (fără spații) între `algoritmi.h:130-172` și `munte-vale.h:29-68` (`sortareInterval`, `sortareIntervalDescrescator`, `sortareMunte`): identice, diferă doar rândurile goale. `cautare-binara.h:15` = `algoritmi.h:45`, doar `mij` e scris altfel. Același lucru la `sortare-interval.h:41` (5.4): corpul e cel din `algoritmi.h:130`, cu un parametru `dim` adăugat și nefolosit.

DE CE: `algoritmi.h` e material de citit, nu de copiat. Un algoritm copiat rulează, dar nu dovedește că îl știi scrie. Urmele copierii se văd și în compilator: `unused parameter 'dim'` la `sortare-interval.h:41`, pentru că ai adăugat un parametru pe care corpul copiat nu-l folosește. Tot așa arată și B3: o funcție copiată de la alta, cu o bucată rămasă din original.

FIX: cu `algoritmi.h` închis, scrii singură sortarea pe interval și `sortareMunte`, apoi compari cu materialul. 5.4 poate refolosi `ordonareCrescator(v, start, stop)` de la 5.3, după ce e reparat B6.

+ același bug la: `munte-vale.h:29` (`sortareInterval`) · `munte-vale.h:47` (`sortareIntervalDescrescator`, copie 1:1) · `munte-vale.h:65` (`sortareMunte`) · `sortare-interval.h:41` (5.4; aceeași funcție apare acum de 3 ori în repo) · `cautare-binara.h:15` (aproape copie; diferă doar `mij`). De verificat cu tine, nu automat 🔴: `sortare-bubble.h:49` (`contorSortareCrescator`, bubble sort adaptat; algoritmul e standard).

🟡 Importante

- **M1** `munte-vale.h:21` — `pozVf` n-are `return` dacă nu găsește vârf (warning `-Wreturn-type`); pe un vector care nu e munte întoarce o valoare oarecare. Pune `return -1;` după buclă.
- **M2** `interclasare.h:125-129` — ramura `else` la care nu se mai ajunge, rămasă din `reuniune` (vezi B3).
- **M3** 7.5 — tot nu există o funcție pentru trei vectori, doar `sol16`.
- **M4** funcții care afișează în loc să întoarcă: `sortare-bubble.h:68`, `sortare-selectie.h:84`, `munte-vale.h:69`, `secventa-egale.h:26`, `:57`, `stergere.h:65` (8.5 nici nu modifică `v` / `dim`). Restanță M1.
- **M5** `frecventa-valori.h:41` — `sol30` are `d = 6` pe un vector cu 4 valori. Restanță M5.
- **M6** vectori de frecvență indexați după valoare: `sortare-selectie.h:37`, `frecventa-valori.h:7`, `:24`. O valoare negativă sau > 999 scrie în afara vectorului. Restanță M4.
- **M7** `secventa-suma.h:13`, `:42` — fereastra glisantă presupune numere pozitive. Restanță M3.
- **M8** `recapitulare-vectori.cpp:3` include `recapitulare-vectori.h`, care nu mai există; fișierul nu e în `.vcxproj`, dar pică la `g++ *.cpp`. Șterge-l sau scoate-l din folder.

🟢 Cleanups

- **C1** `sortare-interval.h:41` — parametrul `dim` e nefolosit (warning).
- **C2** `munte-vale.h:16`, `:109` — `esteVarf(...) == 1` se scrie `if (esteVarf(...))`. Restanță C2.

- **C3** `pozitie-secventa.h:15` — `return j == dy;`. Restanță C3.
- **C4** `munte-vale.h:82` — `//todo` rămas. Restanță C4.
- **C5** `Source.cpp` — peste 50 de apeluri comentate. Restanță C5.
- **C6** `sortare-bubble.h:6` — `nrPar` e `return n % 2 == 0;`. Restanță C6.
- **C7** `suma-maxima.h:4` — comentariul `//13.5` stă deasupra lui 13.4. Restanță C1.

Before / After

#	Acum	Corect
B1	`sol15` cheamă `interclasare(...)`	`sol15` cheamă `reuniune(...)`
B2	`if (c[dc] = a[i]) { dc++; i++; j++; }`	`c[dc] = a[i]; dc++; i++; j++;`
B3	cozi cu garda `c[dc-1] != a[i]`	`while (i < da) c[dc++] = a[i++];`
B4	garda doar în cozile lui `reuniune`	garda înainte de fiecare `c[dc++]` din `reuniune`
B5	`for (i = 1; ... ok == -1 ...)`	`for (i = 0; i < dim; i++) if (v[i] < max && v[i] > ok) ok = v[i];`
B6	`ordonareCrescator(v, 0, mij); ordonareDescrescator(v, mij, dim);`	`(v, 0, mij - 1)` și `(v, mij, dim - 1)`
B7	`cautareBinara(v, d, x)` pe `30 12 45 7 19`	`sortare(v, d);` apoi căutarea
B8	`int mediAritmetica(...)`, `int media = ...`	`double mediAritmetica(...)`, `double media = ...`
B9	`sortareMunte` copiată din material	scrisă de tine, cu materialul închis

Întrebări

1. La B1, de ce `#include "interclasare.h"` nu face ca toate funcțiile din fișier să fie cunoscute dintr-odată? Ce se schimbă dacă muți definiția lui `interclasare` deasupra lui `sol15`? Și de ce tot n-ar fi corect `sol15`?
2. La B2, ce valoare are expresia `c[dc] = a[i]` când `a[i]` e 7? Dar când e 0? Explică de ce la 0 bucla `while` nu se mai termină.
3. Ai acum `reuniune` (7.4) și `interclasare` (7.5). La care dintre ele e nevoie de garda `c[dc-1] != x` și în ce locuri? Alege un test cu dubluri în interiorul aceluiași vector care să le deosebească.

Runda 2 — 2026-10-08 — sha bbd61db

Verificată prin rulare în harness; codul, așa cum era, nu compila.

- 🔴 B1 copiere din `algoritmi.h` — NEATINS (runda 3 B9; în plus `sortare-interval.h:41`)
- 🟡 B2 7.5 moștenea modificarea din 7.4 — PARȚIAL: 7.4 e separat în `reuniune` ✅, dar `interclasare` încă pierde dubluri (runda 3 B3)
- 🔴 B3 7.4 dubluri doar la coadă — NEATINS: `7.4 a:2 2 2 b:3 -> 2 2 2 3` (runda 3 B4)
- 🔴 B4 1.4 — rescris, tot greșit: `7 2 9 4 9 -> 2` (runda 3 B5)
- 🔴 B5 căutare binară pe vector neordonat — NEATINS (runda 3 B7)
- ✅ B6 2.4 `>` vs `>=` — închis: `2 5 9 14 x=9 -> 9`
- 🔴 B7 8.4 împărțire întreagă — NEÎNCHIS: calculul e în `double`, `return`-ul e `int` (runda 3 B8)
- ✅ B8 10.5 condiție copiată — închis: `6 4 2 -> 3`, `5 5 5 -> 3`
- ✅ B9 8.5 impar — închis: `2 2 2 5 5 -> 2 2 2` (tot afișează în loc să șteargă, vezi M4)
- ✅ B10 14.5 santinela 0 — închis: `0 0 1 -> 0`
- ✅ M2 `sol12` afișa înainte de sortare — închis: `9 1 3 5 7 2`

Runda 1 — 2026-10-01 — sha 1e854cc

`recapitulare-vectori.h` → redenumit `parcurgerea-vectoriilor.h`, **neatins de atunci** (100% identic) și scos din build. Constatările rămân valabile:

- 🔴 B1 `sol17` `2 * m -1/ 7` (precedență) — NEATINS

- 🔴 B2 `sol13` poziție vs indice, `v[12]` — NEATINS
- 🔴 B3 `while (n > 10)` la `UltimaPrimaCif` + `cifegale` — NEATINS
- 🔴 B4 `cifMinima(0)` = `9` — NEATINS
- 🔴 B5 `sol19` ignoră `m` — NEATINS
- 🔴 B6 `cmmdc(n, 0)` buclă infinită — NEATINS
- 🔴 B7 `cifreMijlocPatrate` fără `return` — NEATINS
- ✅ B8 santinela `9999` — închis în codul nou (`minVector` pornește din `v[0]`); în fișierul vechi `minElemVec` e neschimbat
- 🔴 B9 `descomp` pe numere prime — NEATINS
- 🔴 B10 `afisareNrRast` modifică vectorul — NEATINS
- 🔴 B11 corpuri în `.h` — NEATINS (acum în 17 headere)
- 🟡 M1–M7 · 🟢 C1–C6 — neatinse; M3 și C4 reapar în runda 2